

Nhóm câu hỏi	STT	Câu hỏi	Câu trả lời
Bitwise	1	Làm thế nào để bật (set) một bit cụ thể trong một biến?	tạo một mặt nạ (mask) có duy nhất bit k là 1: 1 << k. Dùng phép OR () giữa x và mask — OR giữ 1 nếu một trong hai bit là 1, nên bit k sẽ thành 1. VD: bật bit k=1 của số thập phân 18 unsigned int x = 18; int k = 0; x = (1u << k);
	2	Làm thế nào để bật (set) một bit cụ thể trong một biến?	*tạo một mặt nạ (mask) có duy nhất bit k là 1: 1 << k. Dùng phép OR () giữa x và mask — OR giữ 1 nếu một trong hai bit là 1, nên bit k sẽ thành 1. VD: bật bit k=1 của số thập phân 18 unsigned int x = 18; int k = 0; x = (1u << k);
	3	Làm thế nào để đảo (toggle) một bit cụ thể trong một biến?	XOR (*) với 1 << k vì XOR có tính: 0*1 = 1, 1*1 = 0, x^0 = x. Công thức: x ^= (1u << k) VD: Đảo biến của 10 unsigned int x = 10; int k = 1; x ^= (1u << k);
	4	Làm thế nào để kiểm tra một bit cụ thể đã được bật hay chưa?	Sử dụng if (x & (1u << k)) sẽ trả về 0 hoặc giá trị nonzero
	5	Làm thế nào để hoán đổi hai số mà không cần biến thứ ba?	Dùng tính chất của XOR: - Nếu a ^= b; b ^= a; a ^= b; thì sau ba bước a và b đổi chỗ
	6	Làm thế nào để kiểm tra một số có phải là lũy thừa của 2 hay không?	Một số (>0) là lũy thừa của 2 ⇔ biểu diễn nhị phân có đúng một bit 1. Khi đó n & (n - 1) == 0, bởi vì - Nếu n có một bit 1 tại vị k, n-1 có bit k = 0 và tất cả bit thấp hơn = 1. AND sẽ => 0. - Nếu n có ≥2 bit 1, n & (n-1) vẫn còn nonzero.
	7	Làm thế nào để đếm số bit 1 trong biểu diễn nhị phân của một số?	sử dụng n & (n-1) xóa 1 bit 1 at a time (rightmost 1). Lặp cho tới khi n = 0 thì dừng
	8	Làm thế nào để kiểm tra một số là chẵn hay lẻ?	Lấy giá trị bit thấp nhất (LSB). Nếu LSB = 1 → lẻ; nếu 0 → chẵn.
	9	Làm thế nào để tìm bù 2 (2's complement) của một số nhị phân?	Sử dụng: ~x trong two's complement = ~x + 1 (bitwise NOT rồi cộng 1).
	10	Làm thế nào để kiểm tra hai số có bằng nhau hay không mà không dùng toán tử so sánh?	Hai số bằng nhau ⇔ XOR = 0. Dùng x ^ y, nếu = 0 thì giống nhau. Để có kết quả boolean 0/1 ta có thể dùng ! (logical NOT in C) hoặc so sánh với 0
	11	Làm thế nào để kiểm tra hai số có bằng nhau hay không mà không dùng toán tử so sánh?	Hai số bằng nhau ⇔ XOR = 0. Dùng x ^ y, nếu = 0 thì giống nhau. Để có kết quả boolean 0/1 ta có thể dùng ! (logical NOT in C) hoặc so sánh với 0
	12	Làm thế nào để tìm bit có trọng số lớn nhất (Most Significant Bit - MSB)	Các phép OR với dịch làm cho tất cả bit thấp hơn MSB thành 1. Sau đó v & ~(v >> 1) giữ lại chỉ bit MSB.
	13	Làm thế nào để tìm bit có trọng số nhỏ nhất (Least Significant Bit - LSB)?	index = __builtin_ctz(n); (count trailing zeros). n phải != 0.
	14	Làm thế nào để tất bit 1 ở ngoài cùng bên phải?	n-1 biến rightmost 1 thành 0 và đặt tất cả bit nhỏ hơn đó thành 1. AND với n sẽ loại bỏ rightmost 1, giữ các bit lớn hơn.
	15	Làm thế nào để cô lập bit 1 ở ngoài cùng bên phải?	- trong two's complement = ~n + 1. Tích n & ~n giữ đúng bit rightmost 1 và đặt các bit khác 0.
	16	Làm thế nào để kiểm tra một số có phải là bội số của 8 hay không?	bội của 8 có ba bit thấp nhất (bit 0,1,2) = 0 → n & 0b111 = 0.
Memory Management	17	Có những phân đoạn vùng nhớ nào?	Có 5 phân vùng chính là: text, data segment, BSS segment, Heap, Stack
	18	Tại sao cần chia bộ nhớ thành nhiều vùng?	<u>Để quản lý hiệu quả và bảo mật:</u> <u>Tách riêng mã, dữ liệu tĩnh, và dữ liệu động giúp:</u> <u>Để bảo vệ vùng code (chỉ đọc, tránh bị ghi đè).</u> <u>Tối ưu cấp phát & thu hồi (Stack và Heap có cơ chế riêng).</u> <u>Cho phép nạp lại từng phần (Memory Mapping, Shared libs).</u> <u>Giúp debug, profiling, và phát hiện lỗi dễ dàng hơn (VD: Segfault báo đúng vùng lỗi).</u>
	19	Biến toàn cục không khởi tạo nằm ở đâu?	Vùng .bss deekhoang tồn dung lượng trong .bin
	20	Hai biến global có cùng giá trị khởi tạo 0 và 10 — tại sao chúng không nằm trong cùng một vùng nhớ?	Vì hệ thống có thể gán 0 mặc định nên sẽ để trong .bss còn khi khác 0 thì cần phải lưu trong file .data
	21	Khi chương trình gọi một hàm lồng nhau nhiều lần (đệ quy), vùng nhớ nào bị ảnh hưởng nhiều nhất?	Stack bị chịu ảnh hưởng nhiều nhất vì mỗi lần gọi hàm hệ thống sẽ tạo thêm stack frame mới
	22	Tại sao biến const thường được đặt trong vùng .rodata thay vì .data?	- Tránh ghi nhầm (bảo mật). - Cho phép chia sẻ vùng nhớ giữa các tiến trình (tiết kiệm RAM). - .data cho phép ghi, nên không an toàn với const
	23	Nếu bạn muốn dữ liệu tồn tại suốt vòng đời chương trình, bạn nên đặt nó ở vùng nhớ nào?	Đặt tại .data hoặc .bss hoặc trên Heap nếu cấp phát động
	24	Tại sao vùng .bss không chiếm nhiều dung lượng trong file .bin, nhưng lại chiếm RAM khi chạy?	- .bss chỉ mô tả kích thước cần dùng, không lưu dữ liệu thực. - Khi nạp chương trình, hệ điều hành cấp RAM và tự động gán 0. - Giúp giảm kích thước file thực thi.
	25	Điều gì xảy ra với Stack khi hàm kết thúc, nhưng biến static trong hàm đó vẫn được giữ giá trị?	Biến x nằm trong .data hoặc .bss, không nằm trên Stack. Nên khi Stack bị thu hồi sau khi hàm kết thúc thì giá trị của x vẫn tồn tại suốt chương trình.
	26	Lỗi Memory Leak xảy ra khi nào? Tại sao? Cách debug.	Nguyên nhân: Do cấp phát vùng nhớ trên Heap nhưng không giải phóng Cách debug: sử dụng Valgrind hoặc AddressSanitizer để tìm vùng nhớ chưa được giải phóng
	27	Lỗi Stack Overflow xảy ra khi nào? Tại sao? Cách debug.	Nguyên nhân: Stack bị đầy do gọi hàm đệ quy quá sâu, cấp phát động quá lớn Cách debug: Sử dụng GDB hoặc giới hạn stack size
	28	Lỗi Segmentation Fault xảy ra khi nào? Tại sao? Cách debug.	Nguyên nhân: - Khi truy cập vùng nhớ không hợp lệ vào null pointer, ghi vào vùng read-only, truy cập phần tử vượt quá mảng Cách debug: - GDB - AddressSanitizer
	29	Lỗi Stack Smashing là gì? Cách compiler phát hiện bằng cơ chế Canary.	Nguyên Nhân: Stack Smashing = Ghi tràn bộ nhớ cục bộ trên Stack --> ghi đè địa chỉ return. Compiler bảo vệ bằng Stack Canary: - Đặt giá trị ngẫu nhiên giữa buffer và return address. - Khi hàm kết thúc, kiểm tra Canary. - Nếu thay đổi — chương trình dừng ngay (phát hiện tấn công).

	30	Lỗi Heap Corruption là gì? Cách phát hiện bằng AddressSanitizer.	Nguyên nhân: - Khi ghi đè vùng nhớ cấp phát trên heap - Free sai vùng nhớ đã cấp phát AddressSanitizer phát hiện lỗi này bằng cách: - Thêm "red zones" giữa vùng nhớ - Báo lỗi ngay khi truy cập vùng cấm.
	31	Lỗi Dangling Pointer là gì? Tại sao nguy hiểm? Cách khắc phục.	Nguyên nhân: - Con trỏ trỏ tới vùng nhớ đã bị free Lý do nguy hiểm: - Có thể ghi vào vùng đã đương dùng lại - Gây hiện tượng crash ngẫu nhiên Cách khắc phục: - Trỏ con trỏ tới NULL - Giải phóng con trỏ
	32	Khi nào nên dùng AddressSanitizer thay vì Valgrind để debug lỗi bộ nhớ?	Khi cần debug code mới, CI/CD, ưu tiên tốc độ
	33	Lỗi Wild Pointer là gì?	Con trỏ chưa được khởi tạo nhưng đã được sử dụng
	34	Các lớp lưu trữ (storage classes) là gì?	Storage class trong C: - vùng nhớ (nơi lưu biến) - thời gian sống (lifetime), - phạm vi truy cập (scope), - liên kết (linkage).
	35	Từ khóa auto dùng để làm gì? Khi nào thì dùng?	auto là storage class mặc định cho biến cục bộ. Nó xác định rằng biến sẽ được tạo khi hàm được gọi và hủy khi hàm kết thúc. Biến nằm trong stack memory.
	36	Từ khóa static dùng để làm gì? Khi nào thì dùng?	Trong hàm (biến cục bộ): - Biến chỉ được khởi tạo một lần duy nhất. - Giữ lại giá trị giữa các lần gọi hàm. - Nằm trong RAM tĩnh (data segment). Ngoài hàm (biến toàn cục hoặc hàm): - Giới hạn phạm vi truy cập trong file hiện tại. - Không thể truy cập từ file khác (ẩn danh tính trong module). Dùng khi: - Cần giữ trạng thái giữa các lần gọi hàm (counter, buffer). - Muốn ẩn biến hoặc hàm trong file hiện tại (tĩnh module).
	37	Từ khóa extern dùng để làm gì? Khi nào thì dùng?	Dùng để khai báo (declare) biến hoặc hàm được định nghĩa ở file khác. Giúp chia sẻ biến toàn cục giữa nhiều file mà không tạo bản sao mới. Dùng khi: - Có nhiều file .c, muốn truy cập chung biến toàn cục. - Nhưng phải đảm bảo chỉ định nghĩa đúng 1 lần (ở file chính).
	38	Từ khóa register dùng để làm gì? Khi nào thì dùng?	Gợi ý compiler lưu biến trong thanh ghi CPU để truy cập nhanh hơn. Không thể lấy địa chỉ (&) của biến register. Dùng khi: - Biến được truy cập cực kỳ thường xuyên (ví dụ biến đếm vòng lặp). - Trong embedded, compiler có thể bỏ qua gợi ý nếu không đủ thanh ghi.
	39	Từ khóa volatile dùng để làm gì? Khi nào thì dùng?	Bảo cho compiler rằng giá trị của biến có thể thay đổi ngoài chương trình (như từ phần cứng hoặc ngắt). Ngăn compiler tối ưu hóa truy cập biến (luôn đọc giá trị thật từ bộ nhớ). Dùng khi: - Biến được cập nhật bởi ISR (Interrupt Service Routine). - Biến ảnh xạ thành ghi phần cứng hoặc port I/O.
Data Types	40	Khác nhau giữa struct và array?	Struct có thể chứa các kiểu dữ liệu khác nhau, truy cập theo tên trường thay vì chỉ số, Các phần tử có kích thước khác nhau
	41	Kích thước struct được tính như thế nào?	Tổng kích thước các trường + padding để căn hàng theo alignment. Compiler thường căn mỗi trường theo kích thước lớn nhất của thành viên.
	42	Căn chỉnh bộ nhớ trong struct hoạt động như thế nào?	Compiler đảm bảo mỗi trường bắt đầu tại địa chỉ chia hết cho kích thước của nó. Ví dụ: trường int (4 byte) phải bắt đầu ở địa chỉ chia hết cho 4. Giúp CPU đọc nhanh hơn (tránh đọc chéo byte boundary).
	43	Padding là gì trong struct? Làm sao tránh padding?	Padding là byte trống chèn vào giữa các trường để căn hàng. Giúp tăng tốc truy cập nhưng tốn bộ nhớ. Tránh padding bằng cách: - Sắp xếp trường từ lớn -> nhỏ - Dùng #pragma pack(1) hoặc __attribute__((packed))
	44	Có thể gán trực tiếp giữa 2 struct cùng kiểu không?	Có 5 phần vùng chính là: text, data segment, BSS segment, Heap, Stack
	45	Struct có thể so sánh bằng toán tử == không?	Không phải so sánh từng trường với nhau hoặc dùng memcmp
	46	Struct có thể dùng làm tham số hàm không?	Có và cả sử dụng con trỏ để tiết kiệm stack
	47	#pragma pack(1) có ý nghĩa gì?	Yêu cầu compiler không chèn padding (căn hàng theo 1 byte). Dùng trong giao tiếp protocol, truyền dữ liệu nhị phân.
	48	Khi truyền struct vào hàm, có thể bị padding sai khi giao tiếp giữa vi điều khiển khác nhau không?	Có thể: - Vì mỗi compiler / vi điều khiển có quy tắc alignment khác nhau. - Dẫn đến dữ liệu bị lệch byte.
	49	Union là gì?	Kiểu dữ liệu đặc biệt cho phép các thành viên chia sẻ cùng vùng nhớ. Chỉ một trường có giá trị hợp lệ tại một thời điểm.
	50	Kích thước của union là bao nhiêu?	Bằng kích thước thành viên lớn nhất, có thể có thêm padding cho alignment.
	51	Union dùng khi nào?	Khi cần tiết kiệm bộ nhớ, hoặc dữ liệu có nhiều cách biểu diễn khác nhau.
	52	Nếu ghi vào một thành viên union rồi đọc thành viên khác thì sao?	Undefined behavior (hành vi không xác định). Tuy nhiên trong embedded đôi khi dùng cố ý để đọc dữ liệu dạng khác (ví dụ ép kiểu bit).

53	Có thể khởi tạo nhiều thành viên của union cùng lúc không?	Không, chỉ được khởi tạo một thành viên duy nhất
54	Union có thể chứa struct không?	Có thể
55	Union thường dùng trong embedded để làm gì?	Dùng để map các thanh ghi phần cứng hoặc giao tiếp dữ liệu (protocol). Cho phép truy cập dữ liệu ở nhiều mức (bit, byte, word).
56	Enum là gì?	Kiểu dữ liệu cho phép đặt tên cho các hằng số nguyên --> giúp code dễ đọc.
57	Mặc định giá trị đầu tiên của enum là gì?	0
58	Enum có thể gán giá trị cụ thể cho phần tử không?	Có thể
59	Giá trị của phần tử tiếp theo nếu không chỉ định?	Là giá trị của phần tử trước + 1
60	Tại sao nên dùng enum thay vì #define?	Enum có kiểu dữ liệu, kiểm tra lỗi biên dịch, Debug dễ. Có thể gán với switch/case
61	Enum có thể âm không?	Có thể